实验报告

数字逻辑与计算机组成实验

lab06：状态机及键盘输入

姓名：　法童

学号：　251250046

邮箱：　251250046@smail.nju.edu.cn

实验时间：　2026-10-12

1 实验目的

1. 掌握 PS/2 的帧格式与采样：把异步的 PS2_CLK、PS2_DATA 同步进 100 MHz 时钟域，在下降沿采样 11 位帧并用奇校验判断合法性。
2. 设计状态机解析扫描码集 2 的通码与断码（F0 前缀），统计按键次数，做到按住不放只算一次。
3. 用 ROM 完成扫描码到 ASCII 的映射，处理 Shift、Caps Lock 的组合关系，输出大写字母与上档符号。
4. 用六位数码管显示键码、ASCII 与按键次数，在 LED 上指示组合键状态。

2 实验原理

2.1 PS/2 帧格式与采样

键盘到主机方向的每一帧是 11 位：1 位起始位（0）、8 位数据位（LSB 先发）、1 位奇校验位、1 位停止位（1）。数据在 PS2_CLK 的下降沿之后更新、在下一个下降沿之前保持，因此在下降沿处采样最稳。板上 PS2_CLK 由键盘驱动，空闲时为高，速率约 10 kHz，远慢于 100 MHz 系统时钟。



图 1: 键盘到主机的 PS/2 帧格式（共 11 位，在 PS2_CLK 下降沿处采样）

PS2_CLK 与系统时钟无关，直接把它当时钟会引入亚稳态，因此先用 clk 对它打三拍同步，再取同步结果里的下降沿作为采样时刻：

```
reg [2:0] ps2_clk_sync;
always @(posedge clk) ps2_clk_sync <= {ps2_clk_sync[1:0], ps2_clk};
wire sampling = ps2_clk_sync[2] & ~ps2_clk_sync[1];    //同步后的下降沿
```

每采到一位移入 buffer，收满 10 位后检查起始位为 0、停止位为 1、^buffer[9:1] 为 1，三项都满足才把 buffer[8:1] 写入 8 深度的 FIFO 并置起 ready。

2.2 扫描码集 2：通码、断码与重发

按下键时先发该键的**通码**；松开时先发固定的前缀 F0、再发该键的通码，两个字节合起来是**断码**。例如 A 的通码是 1C，松开 A 时依次收到 F0、1C。”按下”占 1 字节、”松开”占 2 字节，长度不同，正好用三个状态描述：IDLE 等一个通码，PRESS 表示该字符键仍被按住，BREAK 在等断码的第二个字节。

按住不放时键盘会周期性重发通码。PRESS 就是用来吸收这些重发的：只有收到 F0 才算松开，中途到达的重发码既不改变显示也不计数。

修饰键不产生字符，它们的通码（12 Shift、14 Ctrl、11 Alt、58 Caps Lock）在 IDLE 中单独识别：Shift / Ctrl / Alt 置位标志，Caps Lock 翻转标志，然后留在 IDLE 等下一个键，不进 PRESS 也不计数；对应的松开（同样是 F0 + 通码）在 BREAK 中清标志。

2.3 扫描码到 ASCII 的映射

字符键的映射用一张 256×8 的 ROM 完成：地址是扫描码、数据是 ASCII 码，初始化时用 $readmemh 读入 scancode.txt；ROM 输出再经第二张表 shift_ascii.txt 得到按住 Shift 时的字符。

Shift 与 Caps Lock 对字母的作用相同，同时按下又互相抵消，而 Caps Lock 不影响数字与符号，因此判定写成：

```
wire shift = (ascii_code >= 8'h61 && ascii_code <= 8'h7a) ? (l_shift
    ^ caps) : l_shift;
```

只有基表给出小写字母（61–7A）时才把 Caps 计入，数字与符号只由 Shift 决定。

3 实验环境与器材

- 开发板：Nexys A7-100T（FPGA 型号 xc7a100tcsg324-1），板载 PS/2 接口
- 开发工具：Vivado 2020.2（非工程模式，build.bat / sim.bat / prog.bat 脚本驱动）
- 硬件描述语言：Verilog HDL
- 输入设备：PS/2 键盘

4 设计思路

设计分三层：PS/2 底层接收（ps2_keyboard）、扫描码解析与映射（keyboard 状态机 + scancode_ram）、显示与指示（hex7seg、display6_4），由 top 接板级端口。

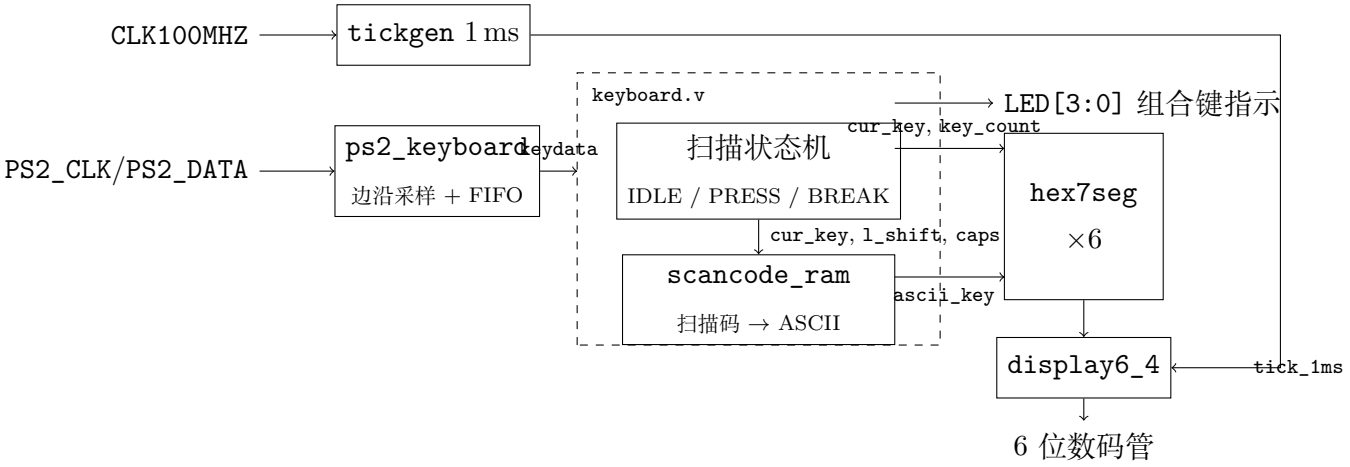


图 2: 系统框图

表 1: 板级资源分配

板级资源	顶层信号	功能
CLK100MHZ	CLK100MHZ	100 MHz 系统时钟（采样与时基）
CPU_RESETN	clrn	复位（低有效）
PS2_CLK、PS2_DATA	同名	PS/2 键盘的时钟与数据
LED[0..3]	l_shift/ctrl/l_alt/caps	组合键状态指示
AN[1:0]	cur_key	当前键码（十六进制，低两位）
AN[4:3]	ascii_key	对应 ASCII 码（十六进制，中间两位）
AN[7:6]	key_count	按键总次数（十六进制，最高两位）
AN[5]、AN[2]	——	恒灭，三组读数之间的空位
CA..CG、DP	段选	段码输出；DP 置 1 熄灭

扫描状态机是本次的核心，三个状态下的动作见下表。计数只发生在 IDLE 收到字符键通码的那一刻，这就是”按住不放只算一次”的实现。

表 2: 扫描状态机的状态转移

状态	收到的 keydata	动作	次态
IDLE	F0	cur_key←0	BREAK
IDLE	12/14/11	置位对应修饰键标志，cur_key←0	IDLE
IDLE	58	caps←~caps，cur_key←0	IDLE
IDLE	其他（字符键通码）	cur_key←keydata，key_count+1	PRESS
PRESS	不等于 F0（长按重发）	保持 cur_key，不计数	PRESS
PRESS	F0	cur_key←0	BREAK
BREAK	12/14/11	清对应修饰键标志	IDLE
BREAK	其他	cur_key←0	IDLE

```
IDLE: case (keydata)
    8'hf0: begin cur_key <= 8'd0; state <= BREAK; end      //断码前缀
    8'h12: begin cur_key <= 8'd0; l_shift <= 1'b1; end     //修饰键只置标志，不计数
    8'h14: begin cur_key <= 8'd0; ctrl    <= 1'b1; end
    8'h11: begin cur_key <= 8'd0; l_alt   <= 1'b1; end
    8'h58: begin cur_key <= 8'd0; caps    <= ~caps; end    //Caps 按一次翻转一次
    default: begin                                         //字符键通码：在这里计数
        cur_key   <= keydata;
        key_count <= key_count + 1;
        state     <= PRESS;
    end
endcase
PRESS: begin
    if (keydata != 8'hf0) cur_key <= keydata;              //重发码：保持显示、不计数
    else begin cur_key <= 8'd0; state <= BREAK; end
end
BREAK: begin                                               //断码的第二个字节
    case (keydata)
```

```
        8'h12: l_shift <= 1'b0;                                    //松开哪位修饰键就清哪个标志
        8'h14: ctrl    <= 1'b0;
        8'h11: l_alt   <= 1'b0;
        default: cur_key <= 8'd0;                                  //字符键松开：清键码
    endcase
    state <= IDLE;
end
```

显示部分把三个字节各拆成两个十六进制半字节交给 hex7seg 译码，再由 display6_4 以 1 ms 节拍轮扫；三组读数各占两位、中间留一格空位：

```
hex7seg hex7seg0(cur_key[3:0],   h0);   //AN[0]：键码低位
hex7seg hex7seg1(cur_key[7:4],   h1);   //AN[1]：键码高位
hex7seg hex7seg2(ascii_key[3:0], h2);   //AN[3]：ASCII 低位
hex7seg hex7seg3(ascii_key[7:4], h3);   //AN[4]：ASCII 高位
hex7seg hex7seg4(key_count[3:0], h4);   //AN[6]：次数低位
hex7seg hex7seg5(key_count[7:4], h5);   //AN[7]：次数高位
```

手册要求”按键松开时低四位全灭”，该状态等价于 cur_key 为 0，用位选掩码实现：

```
wire dis4 = (cur_key != 8'd0);
assign mask = dis4 ? 8'b00000000 : 8'b00011011;   //松开时熄灭 AN[0]/[1]/[3]/[4]
```

5 实验过程

1. ps2_keyboard 完成 PS2_CLK 的同步与边沿采样、奇校验判断与 8 深度 FIFO，并按 ready/nextdata_n 握手输出扫描码。

2. 编写 keyboard.v 的扫描状态机（IDLE/PRESS/BREAK），输出 cur_key、key_count 与四个修饰键标志；同文件内例化 scancode_ram。

3. 整理两张 ROM 表：scancode.txt 给出各扫描码的基础字符，shift_ascii.txt 给出按下 Shift 后的字符。

4. 编写 top.v：复用 lab03 的 tickgen 产生 1 ms 扫描节拍、lab04 的 hex7seg，6 个实例译码后交给 display6_4 选通六位。

5. 从根目录 nexysa7.xdc 复制约束,取消 CLK100MHZ、CPU_RESETN、PS2_CLK、PS2_DATA、LED[3:0]、AN[7:0]、CA..CG、DP 的注释。

6. 运行 build.bat lab06\keyboard 生成比特流、prog.bat 下载上板;编写 sim/tb_top.v,用 sim.bat 跑仿真。

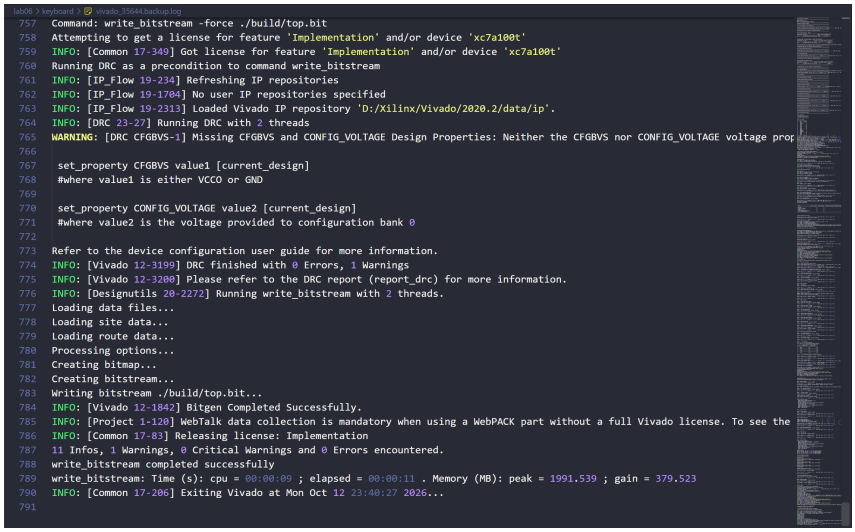


图 3: 综合、实现与比特流生成完成

6 测试方法

如何验证：按键的”按下 → 按住 → 松开”由人手在板上产生，长短与重发时刻都不可复现，”松开序列只收到一个 F0 就中断”这类边界更难制造，因此时序相关的验证交给仿真：测试台按 PS/2 帧格式逐位驱动 PS2_CLK 与 PS2_DATA 造出按键序列，再比对 key_count、cur_key、ascii_key 与四个修饰键标志。上板只验证与硬件相关的部分：键盘对接、译码显示与 LED 指示。

测试信号的选择按”覆盖三种帧形态 + 覆盖修饰键的两种时序 + 覆盖边界”来设计：

- **基本要求**：按 A（通码 1C）后次数为 1、键码 1C、ASCII 61；再发一次 1C 模拟长按重发，读数不变；发 F0、1C 模拟松开，键码与 ASCII 清零；再按一次 A，次数为 2。三种帧形态各覆盖一次，即”按住不放只算一次”。
- **Shift 与字符键**：按 12（只点亮 LED、不计数），再按 A 应得 41（大写 A）。
- **Shift 与数字**：按 16 应得 21（上档符号!）。

- **Caps Lock 与边界**：按一次 58 翻转 caps，再按翻回；caps 下按 A 得 41；caps 与 Shift 同按得 61（抵消）；caps 下按数字 16 仍得 31，即 Caps 不影响数字。
- **Ctrl 与 Alt**：只点亮 LED，不计入次数。

（mod 按 {Shift, Ctrl, Alt, Caps} 的顺序给出 4 位状态。）**上板测试**：接上键盘，按 CPU_RESETN 复位后依次按下若干字母、数字与组合键，观察三组读数与 LED 指示。

7 实验结果

7.1 仿真

测试台共 44 项检查全部通过，摘录如下（每项在帧结束后等 ascii_key 的两拍寄存器延迟再比对）：

```
press A (1C)
  pass: cnt=1 key=1c asc=61 mod=0000
hold A (1C) #2
  pass: cnt=1 key=1c asc=61 mod=0000      <- 长按重发码不计数
break F0
  pass: cnt=1 key=00 asc=00 mod=0000
release A (1C)
  pass: cnt=1 key=00 asc=00 mod=0000      <- 松开后键码与 ASCII 清零
press A (1C) again
  pass: cnt=2 key=1c asc=61 mod=0000      <- 再按一次才累加
press LShift (12)
  pass: cnt=2 key=00 asc=00 mod=1000      <- Shift 只置标志、不计数
press A (1C)
  pass: cnt=3 key=1c asc=41 mod=1000      <- Shift+A -> 大写 41
press 1 (16)
  pass: cnt=4 key=16 asc=21 mod=1000      <- Shift+1 -> 上档符号 21
press Caps (58)
  pass: cnt=4 key=00 asc=00 mod=0001      <- Caps 翻转
press A (1C)
  pass: cnt=5 key=1c asc=41 mod=0001      <- Caps 下字母大写
press LShift (12)
  pass: cnt=5 key=00 asc=00 mod=1001      <- Caps 与 Shift 同时按下
press A (1C)
  pass: cnt=6 key=1c asc=61 mod=1001      <- 两者抵消，回到小写
```

```
press LCtrl (14)
  pass: cnt=7 key=00 asc=00 mod=0100
press LAlt (11)
  pass: cnt=7 key=00 asc=00 mod=0010
==== ALL PASS ====
```

长按不重复计数、松开清零、Shift 与 Caps 的组合关系、修饰键只置 LED 这几点都得到验证，与手册 6.4 的要求一致。

7.2 上板

上板实测现象与设计一致：

- **基本显示**：按一次 `A`，右侧两位显示键码 `1C`、中间两位显示 ASCII `61`、最高两位次数为 `01`；松开后键码与 ASCII 四位全灭，只剩次数。
- **计数**：按住 `A` 不放次数不再增加，松开后再按才变成 `02`；连按不同键逐个累加。
- **组合键**：按住 Shift 时 `LED[0]` 亮，此时按 `A` 显示 `41`，按数字 1 显示 `21`。
- **Caps Lock**：按一次 `LED[3]` 亮、此后字母显示大写，再按一次熄灭、回到小写；Caps 与 Shift 同按时又回到小写，且不影响数字键。
- **Ctrl / Alt**：按下时 `LED[1]`、`LED[2]` 分别点亮，次数不增加。



图 4: 上板实测：键码、ASCII 与按键次数

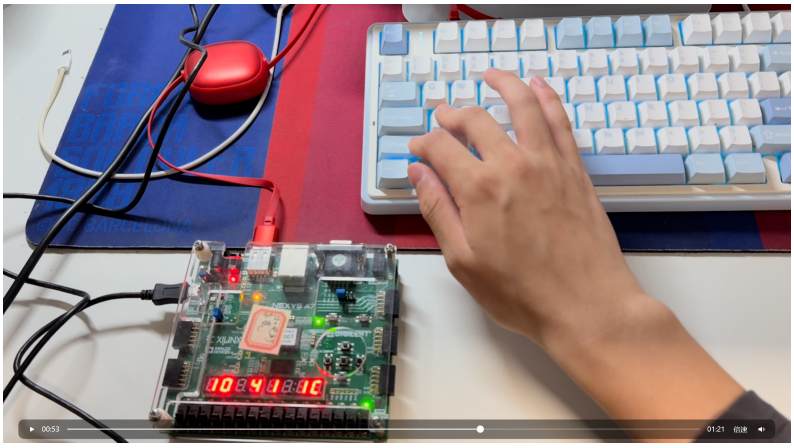


图 5: 上板实测：Shift+A 显示大写，LED[0] 指示



图 6: 上板实测：Caps 翻转，松开后低四位全灭

7.3 综合后的 Schematic

综合后的网表层次与设计框图一致：`top` 下面是 `tickgen`、`keyboard`、6 个 `hex7seg` 与 `display6_4`，`keyboard` 下面是 `ps2_keyboard`、`scancode_ram` 与状态机本身的寄存器。两处存储的映射方式不同：`scancode_ram` 的两张 256×8 字符表是同步读的，被综合到一块 Block RAM 硬核（`RAMB18E1`）；`ps2_keyboard` 里 8 深度 $\times$ 8 位的 FIFO 只有 64 位，走的是 LUT 搭出来的分布式 RAM（`RAMD32`）。

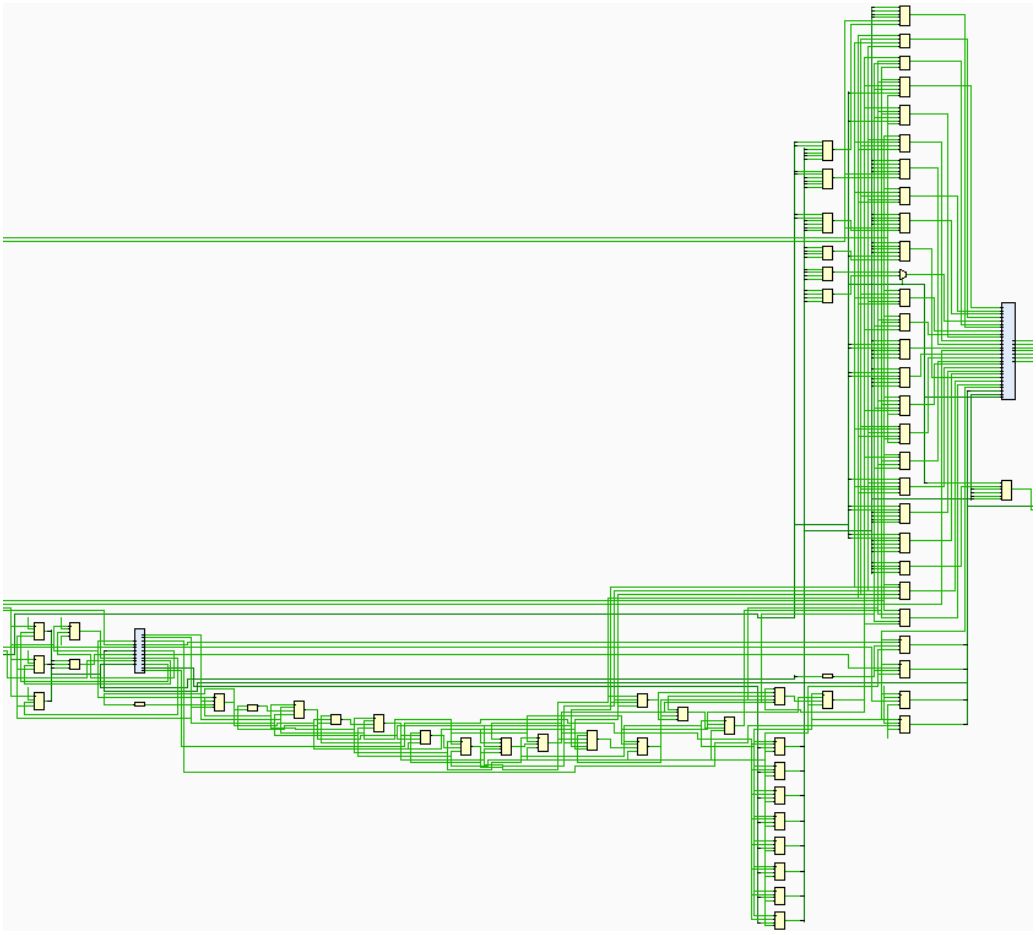


图 7: 综合后的 Schematic：顶层与 `keyboard` 内部的层次

8 问题与解决办法

- **长按被重复计数**：最初收到通码就计数并回到 `IDLE`，而键盘按住不放会周期性重发同一个通码，于是一次按住被计成很多次。原因是没有区分”键仍被按住”这一情形。解决：增加 `PRESS` 状态，只有从它收到 `F0` 才回到 `IDLE`，中途的重发码不计数也不清键码。

- **修饰键被当成普通按键**：最初不识别 12/14/11/58，按下 Shift 被当作字符键计数、还显示出 12 这样的” 键码”，松开时 F0+12 又被算一次。解决：在 IDLE 中单独 case 这四个扫描码，只置/翻转标志并留在 IDLE；对应的松开在 BREAK 中用同一个扫描码清标志。两处都要改，否则会出现 Shift 松开后 LED 一直亮着。

- **Caps Lock 的语义与 Shift 不同**：Caps 是” 按一次翻转一次”，若照搬 Shift 的” 按住置位、松开清零”，重发码会让标志反复翻转、松开后又清零。解决：在 IDLE 收到 58 时做 caps <= ~caps，而 F0+58 在 BREAK 中不修改 caps，这样一次按下恰好翻转一次。

- **位选重排后熄灭掩码没跟着改**：位选从连续四位改成 AN[1:0]/AN[4:3]/AN[7:6]、组间各留一格空位后，” 低四位全灭” 不再是连续的低四位，掩码要从 8'b00001111 改成 8'b00011011；否则会灭掉次数的最低位、却在 ASCII 低位留下一个 0。

- **ascii_key 有两拍延迟**：scancode_ram 先寄存基字符、再寄存移位后的结果，共两拍，测试台若在帧结束的当拍比对会读到旧值。解决：比对前多等两拍。这类问题只在仿真中暴露，上板看不出来。

9 启示与建议

启示：

1. 一开始，我没有涉及明确的状态机模式，比如状态空间、转移方程等详细内容，而是凭借经验自行编写一些逻辑，编写和调试起来都缺乏条理。后来我重构了代码逻辑，设计了详细的状态机，运用 IDLE, PRESS, BREAK 三个状态，并梳理相应的转移策略。这样一来时序逻辑、代码结构都变清晰了。我由此认识到状态机的优越性。

2. 外部设备的时钟不能直接用来驱动本地逻辑：PS2_CLK 与 100 MHz 主时钟无关，先用三级同步器把它带进本地时钟域、再取边沿采样，才不会出现亚稳态。

3. 扫描码把” 按下” 和” 松开” 编成长度不同的字节序列，再加上长按重发，正好用三个状态对应” 等新键”” 键仍按住”” 等断码的第二个字节”，把时间上的先后关系拆成了状态转移。

建议：单个按键逐个按下松开的过程很容易理解。但涉及组合键、多个按键按不同顺序按下松开，此时的 PS/2 数据流比较复杂，有点难判断。建议手册上适量添加组合键按下松开过程中 PS/2 数据流变化过程的详细说明。